

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание №4

Выполнил:

Майстренко Анастасия Николаевна

K3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Спроектировать разделение монолитного бэкенд-приложения сервиса аренды недвижимости на микросервисы по принципу database-per-service. Необходимо:

- выделить микросервисы и описать их взаимосвязи и способы взаимодействия;
- спроектировать разделение единой БД на обособленные БД сервисов;
- спроектировать межсервисные эндпоинты в формате OpenAPI с описанием запросов, ответов и ошибок;
- сформировать документ с требованиями и шагами перехода от монолита к микросервисам.

Ход работы

1. Выделение микросервисов

Монолит из ЛР1 проанализирован, и его сущности сгруппированы по ограниченным контекстам (bounded contexts) предметной области. Каждый контекст становится отдельным микросервисом со своей зоной ответственности (таблица 1). Связанность внутри сервиса высокая, между сервисами — низкая.

Таблица 1 — Состав микросервисов

Сервис	Зона ответственности	Сущности
Identity Service	Регистрация, вход, JWT, профили, роли	User
Catalog Service	Объекты, поиск, фото, удобства, избранное	Property, PropertyPhoto, Amenity, Favorite
Booking Service	Бронирования, статусы сделок, отзывы	Booking, Review
Messaging Service	Диалоги и сообщения между пользователями	Conversation, Message

Перед сервисами размещается API Gateway — единая точка входа, которая выполняет маршрутизацию запросов, проверку JWT и агрегацию ответов нескольких сервисов (API Composition).

2. Разделение базы данных (database-per-service)

Единая БД монолита разделяется на обособленные БД — по одной на сервис. Прямые внешние ключи между таблицами разных сервисов недопустимы: вместо них хранятся идентификаторы (например, Booking

хранит `propertyId` и `tenantId` как обычные числа без FK). Сопоставление БД и таблиц приведено в таблице 2.

Таблица 2 — Разделение БД по сервисам

Сервис	База данных	Таблицы
Identity	users_db	user
Catalog	catalog_db	property, property_photo, amenity, property_amenity, favorite
Booking	booking_db	booking, review
Messaging	messaging_db	conversation, message

Поскольку соединения (JOIN) между БД невозможны, применяются два приёма: денормализация (Booking хранит снимок названия и цены объекта на момент брони) и обогащение данных через межсервисные запросы или асинхронные события. Это обеспечивает слабую связанность и независимое масштабирование и развёртывание сервисов.

3. Способы взаимодействия сервисов

Используются два типа взаимодействия:

Синхронное (REST). Сервис вызывает внутренний (`/internal`) эндпоинт другого сервиса, когда данные нужны немедленно. Например, Booking при создании брони запрашивает у Catalog цену и владельца объекта, а у Identity — проверку токена. Внешние клиентские запросы проходят через API Gateway.

Асинхронное (события). Сервисы публикуют события в брокер сообщений (RabbitMQ/Kafka), а заинтересованные сервисы на них подписываются. Это используется там, где допустима итоговая согласованность (eventual consistency). Основные события приведены в таблице 3.

Таблица 3 — Доменные события

Событие	Издатель	Подписчики и реакция
user.registered	Identity	Catalog/Messaging — кэш базовых данных пользователя
property.updated	Catalog	Booking — обновление снимка цены/названия
booking.confirmed	Booking	Catalog — статус объекта rented; Messaging — открыть диалог
booking.completed	Booking	Catalog — статус available; разблокировка отзыва
booking.cancelled	Booking	Catalog — возврат статуса available

Для распределённой бизнес-операции бронирования применяется паттерн Saga: последовательность шагов (проверка объекта → создание

брони → смена статуса объекта) с компенсирующими действиями при сбое. Запросы делаются идемпотентными (ключ идемпотентности), чтобы повторы не приводили к дублированию.

4. Схема архитектуры

Итоговая схема микросервисной архитектуры приведена на рисунке 1. На ней показаны клиент, API Gateway, четыре сервиса со своими БД, брокер сообщений, а также синхронные (REST) и асинхронные (события) связи. Исходник схемы — файл microservices-architecture.drawio.

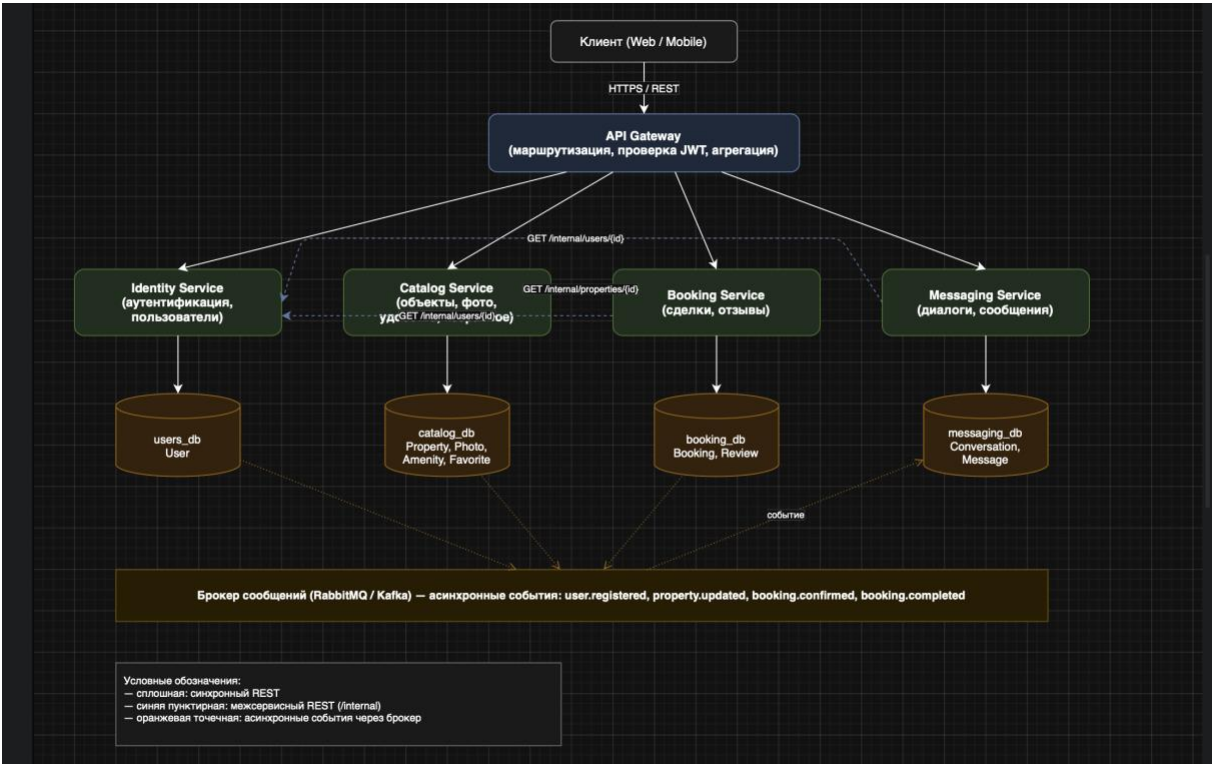


Рисунок 1 — Микросервисная архитектура сервиса аренды недвижимости

5. Межсервисные эндпоинты (OpenAPI)

Внутренние эндпоинты для межсервисного взаимодействия описаны в OpenAPI-спецификации (файл internal-api.openapi.yaml, стандарт OpenAPI 3.0.3, прошла валидацию). Эндпоинты не публикуются наружу через Gateway и защищены служебным токеном (заголовок X-Internal-Token) либо mTLS. Перечень — в таблице 4.

Таблица 4 — Межсервисные эндпоинты

Сервис	Эндпоинт	Назначение
Identity	POST /internal/tokens/verify	Проверка JWT, возврат userId и роли
Identity	GET /internal/users/{id}	Данные пользователя для обогащения

Identity	POST /internal/users/batch	Пакетная выборка пользователей (против N+1)
Catalog	GET /internal/properties/{id}	Цена, владелец, статус объекта
Catalog	PATCH /internal/properties/{id}/status	Смена статуса объекта (rented/available)
Booking	GET /internal/bookings/{id}	Данные брони
Booking	GET /internal/bookings/exists	Проверка завершённой брони (право на отзыв)
Messaging	POST /internal/conversations	Служебное создание диалога

Запросы и ответы передаются в формате JSON. Например, *GET /internal/properties/{id}* возвращает объект с полями `id`, `ownerId`, `title`, `pricePerDay` и `status`. Все тела запросов и ответов, а также схемы данных формализованы в файле спецификации.

6. Обработка ошибок и отказоустойчивость

Ошибки возвращаются в едином формате — объект с полями `code` и `message`. Используемые коды: 401 (недействительный служебный токен или JWT), 404 (ресурс не найден), 409 (конфликт состояния, например недопустимый переход статуса), 422 (ошибка валидации), 503 (сервис недоступен). Для устойчивости межсервисного взаимодействия применяются:

- таймауты и повторные попытки (retry) с экспоненциальной задержкой;
- предохранитель (circuit breaker) — отключение вызовов к недоступному сервису;
- идемпотентность операций, чтобы повторные запросы не дублировали данные;
- итоговая согласованность через события там, где немедленная консистентность не требуется.

7. Шаги перехода от монолита к микросервисам

Переход выполняется поэтапно по стратегии Strangler Fig (постепенное «удушение» монолита), без полной остановки сервиса:

1. Выделить в монолите модули по ограниченным контекстам и развязать прямые зависимости между ними через интерфейсы.
2. Заменить межмодульные внешние ключи на хранение идентификаторов и денормализованные снимки данных.
3. Поднять API Gateway и перенести на него маршрутизацию и проверку JWT.

4. Первым выделить Identity Service с отдельной БД users_db (аутентификация нужна всем).
5. Поочерёдно выделить Catalog, Booking и Messaging, каждый со своей БД, перенося данные.
6. Внедрить брокер сообщений и перевести часть межсервисных взаимодействий на асинхронные события.
7. Реализовать /internal-эндпоинты для синхронных запросов и настроить служебную аутентификацию.
8. Настроить отказоустойчивость (retry, timeout, circuit breaker), наблюдаемость (логи, трейсинг, метрики) и независимый CI/CD для каждого сервиса.

Вывод

В ходе работы спроектировано разделение монолитного бэкенд-приложения сервиса аренды недвижимости на четыре микросервиса (Identity, Catalog, Booking, Messaging) по принципу database-per-service. Описаны зоны ответственности сервисов, разделение единой БД на обособленные базы, а также синхронный (REST через /internal-эндпоинты) и асинхронный (события через брокер сообщений) способы взаимодействия.

Межсервисные эндпоинты формализованы в виде валидной OpenAPI-спецификации с описанием запросов, ответов и ошибок; архитектура представлена схемой. Сформированы конкретные требования и пошаговый план миграции от монолита к микросервисам по стратегии Strangler Fig, что обеспечивает слабую связанность, независимое развёртывание и масштабируемость системы.